

Assignment 3

Task 1: Basic Doubly Linked List Structure

Task:

1. Define a structure for a doubly linked list node that holds an integer value, a pointer to the next node, and a pointer to the previous node.
2. Implement a function to add a node to the beginning of the list.
3. Implement a function to add a node to the end of the list.

Sample Input:

- Add 5 to the beginning.
- Add 10 to the end.
- Add 15 to the end.

Sample Output: Linked List: 5 <-> 10 <-> 15

Task 2: Extended Doubly Linked List Operations

Task:

1. Implement a function to insert a node after a specific value.
2. Implement a function to insert a node at a given position.
3. Implement a function to delete a node with a given value.
4. Implement a function to delete a node at a given position.

Sample Input:

- Insert 25 after 10.
- Delete the node with the value 10.
- Insert 20 at position 2.
- Delete the node at position 3.

Sample Output: Linked List: 5 <-> 25 <-> 20

Task 3: Reversing the Doubly Linked List

Task:

1. Implement a function that reverses the order of nodes in the list in-place.

Sample Input:

- Reverse the linked list.

Sample Output: Linked List: 20 <-> 25 <-> 5

Task 4: Cycle Detection in a Doubly Linked List**Task:**

1. Implement a function to check if a doubly linked list contains a cycle.
2. Implement a function to find the start node of the cycle if it exists.

Sample Input:

- Create a linked list with a cycle.

Sample Output:

- Has Cycle: Yes
- Cycle Start Node: 10

Task 5: Merging Two Sorted Doubly Linked Lists**Task:**

1. Implement a function to merge two sorted doubly linked lists into one sorted list. Ensure that the head pointers of the lists are updated correctly.

Sample Input:

- Merge two sorted lists: List A: 5 <-> 10, List B: 7 <-> 12.

Sample Output: Merged List: 5 <-> 7 <-> 10 <-> 12

These tasks cover the basic and advanced operations for working with doubly linked lists, ensuring that you implement and understand insertion, deletion, traversal, and complex operations like reversal, cycle detection, and merging.

Task 6: Finding the Length of a Doubly Linked List

Task:

1. Implement a function to find the length of the doubly linked list.

Sample Input:

- Current List: 5 <-> 10 <-> 15

Sample Output:

- Length: 3

Task 7: Searching for a Value in the Doubly Linked List

Task:

1. Implement a function to search for a specific value in the list and return its position.

Sample Input:

- Search for 10 in the list 5 <-> 10 <-> 15

Sample Output:

- Value 10 found at position 1

Task 8: Sorting the Doubly Linked List

Task:

1. Implement a function to sort the doubly linked list in ascending order.

Sample Input:

- Current List: 15 <-> 5 <-> 10

Sample Output:

- Sorted List: 5 <-> 10 <-> 15

Task 9: Removing Duplicates from the Doubly Linked List Task:

1. Implement a function to remove duplicate values from the list.

Sample Input:

- Current List: 5 <-> 10 <-> 10 <-> 15

Sample Output:

- List after removing duplicates: 5 <-> 10 <-> 15

Task 10: Splitting the Doubly Linked List

Task:

1. Implement a function to split the doubly linked list into two halves. Ensure that the two halves are returned as separate lists.

Sample Input:

- Current List: 5 <-> 10 <-> 15 <-> 20

Sample Output:

- First half: 5 <-> 10
- Second half: 15 <-> 20

Task 11: Palindrome Check

Task:

1. Implement a function to check if the doubly linked list is a palindrome (reads the same forward and backward).

Sample Input:

- Current List: 1 <-> 2 <-> 3 <-> 2 <-> 1

Sample Output:

- Is Palindrome: Yes

Task 12: Swapping Nodes

Task:

1. Implement a function to swap two nodes in the doubly linked list without swapping the data.

Sample Input:

- Swap nodes with values 10 and 15 in the list 5 <-> 10 <-> 15

Sample Output:

- List after swapping: 5 <-> 15 <-> 10

Task 13: Finding the Middle Node

Task:

1. Implement a function to find the middle node of the doubly linked list.

Sample Input:

- Current List: 5 <-> 10 <-> 15

Sample Output:

- Middle Node: 10

Task 14: Rotating the List

Task:

1. Implement a function to rotate the doubly linked list by a given number of positions.

Sample Input:

- Rotate the list 5 <-> 10 <-> 15 by 2 positions.

Sample Output:

- Rotated List: 15 <-> 5 <-> 10

Task 15: Copying the Doubly Linked List

Task:

1. Implement a function to create a deep copy of the doubly linked list.

Sample Input:

- Current List: 5 <-> 10 <-> 15

Sample Output:

- New List (copy): 5 <-> 10 <-> 15

These tasks will help you master various operations on doubly linked lists, from basic manipulations to more advanced algorithmic challenges.